



Draw It or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	4
Evaluation	5
Recommendations	8

Document Revision History

Version	Date	Author	Comments
1.0	01/24/26	Robert Cardoza	Initial version of the software design document and game application implementation.
1.0	02/07/26	Robert Cardoza	Evaluated client and server sided considerations for development, along with tools that available for different operating systems.
1.0	02/21/26	Robert Cardoza	Added final platform recommendation, storage and memory strategy, distributed architecture and security considerations.

Executive Summary

The Gaming Room is expanding its existing Android game, *Draw It or Lose It*, into a web-based, multi-platform application that supports multiple concurrent games, teams, and players. To support scalability and consistent behavior across distributed environments, the design uses an object-oriented domain model with shared entity behavior and unique identifiers across all core objects. A singleton service (GameService) is used to ensure only one game service instance exists in memory, centralizing creation and lookup of games while also managing unique identifiers for games, teams, and players. Uniqueness of game and team names is enforced using iteration-based lookups prior to object creation, preventing duplicate naming collisions and supporting user-friendly name availability checks.

Requirements

The web version of *Draw It or Lose It* at The Gaming Room needs to be able to accommodate several games at once while still acting consistently and predictably on all platforms. One or more teams, each with several players, must be able to participate in a game using the application. All game and team names must be distinct in order to facilitate fair gameplay and user clarity. This allows users to confirm name availability prior to joining or forming a game or team.

Technically speaking, in order to consolidate the creation and management of games, teams, and players, the system must guarantee that there is only one instance of the game service in memory at any given time. To avoid confusion and provide dependable lookup and tracking within the system, each game, team, and player must be given a distinct identifier. To prevent duplicate entries and inconsistent states, the application must programmatically enforce uniqueness rules prior to object creation. Lastly, the design must maintain acceptable performance and responsiveness during timed gameplay while supporting future scalability and deployment in a distributed, web-based environment.

Design Constraints

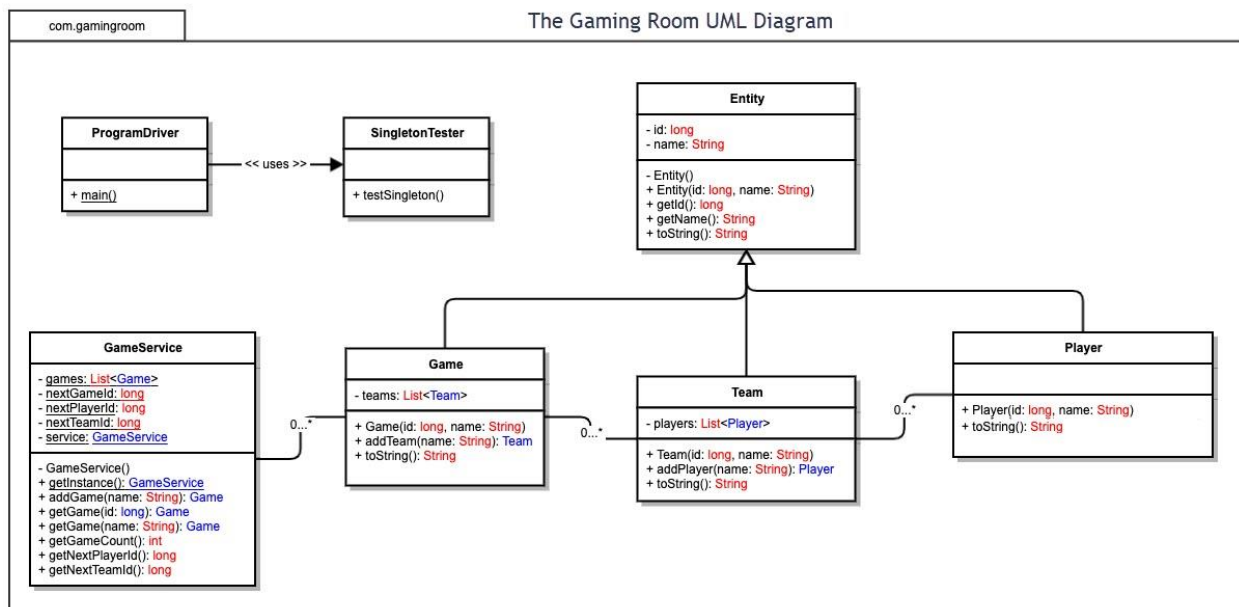
A web-based distributed environment imposes a number of design constraints that have an impact on the application architecture. First, several users may attempt to create games, teams, or players simultaneously; their attempts should be carefully managed in order to avoid duplicate entries or introduce inconsistent states. Second, the system has to be able to support multiple platforms, which means the design should prefer portable server-side logic and prohibit making platform-specific assumptions. Third, given that the application could be horizontally scaled (multiple servers/instances), any design relying solely upon in-memory storage will be limited by consistency across deployments. Eventual deployment to production would require persistent storage or a centralized state management layer to handle such issues. Finally, performance and responsiveness are inescapable issues, because gameplay is timed with round timers and guessing windows. Object creation and lookups need to be fast and predictable. Such constraints influence a design toward the central management of object creation, a clear separation of responsibilities, and systematic enforcement of uniqueness rules.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML domain model shows the game as a hierarchy of related entities. The Entity base class defines shared attributes like id and name, along with common behaviors such as getters and toString(). Game, Team, and Player inherit from this class. This setup shows inheritance and reuse, which reduces duplication and keeps identity management consistent across object types. A Game holds a collection of Team objects, creating a one-to-many relationship. Each Team also contains a collection of Player objects, forming another one-to-many relationship. The model uses encapsulation by keeping contents private and controlling creation through methods like addTeam() and addPlayer(). These methods enforce uniqueness rules before adding objects. The GameService class manages active games, using the singleton pattern to ensure that only one instance exists in memory. This meets the requirement that only one game service instance exists at any time. The iterator pattern is used throughout the model—in addGame, addTeam, and addPlayer—to search existing collections before creating new instances. This ensures that game, team, and player names stay unique and allows for name availability checks.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can be used to host a web-based application with the help of various server technologies like Apache, Nginx, or application servers running on Java Virtual Machines. Moreover, this is due to its Unix base, providing stability and also great open-source server toolset support. However, macOS is not considered for large-scale server operations for hosting web-based applications, as it is costly in comparison to other server technologies, as it requires Apple hardware for hosting and other related costs. Although it can be used for hosting, scaling up for thousands of concurrent users is not feasible.	Linux is currently the leading operating system for web-based applications hosting in distributed environments. The current dominance of Linux is due to its server-based hosting options, performance, and excellent scalability features, which can support many concurrent users up to thousands. Linux has many options for open-source web servers, databases, and containerization software, which makes it perfect for hosting environments, data centers, and hosting environments. Linux is also free or very cheap, which makes it very appropriate for The Gaming Room since it can be acquired at a very low cost. Linux is also the strongest option for hosting web-based applications.	Windows Server is a mature technology and offers server-side technology along with server-side deployment. It is also compatible and works well for web application hosting as it also provides IIS. Windows Server is a highly scalable technology as long as it is deployed correctly. Although Windows server is a good choice as server-side, for web application hosting one of the negative aspects is related to the cost aspect as it tends to be more expensive compared to running a Linux server.	Mobile devices are not meant to be used to host the server-side web application. Hence, it may be concluded that it is not feasible to deploy the servers on mobile devices. To host a centralized and scalable web server, its implementation would be quite difficult due to the lack of processing power, memory issues, and reliability issues of the mobile operating system as well as other issues. In the proposed architecture, the mobile devices are used as the client, which communicates with the backend servers deployed on the traditional operating systems or platforms such as Linux, Windows, and macOS.

Client Side	From the perspective of the clients, the macOS operating system has good support for modern web browsers such as Safari, Chrome, and Firefox. This allows the client to access the game by using a responsive interface created by HTML. The considerations related to the development involve ensuring the recent browsers and responsive interface for the clients. The costs for the clients using the macOS operating system involve moderate development related to hardware.	Linux clients can access the application through supported web browsers such as Firefox and Chrome. While Linux distributions vary, adherence to web standards ensures consistent behavior across environments. Development considerations include browser compatibility testing rather than operating system-specific changes. Supporting Linux clients does not significantly increase development cost or time, provided the application is built using standard HTML, CSS, and JavaScript practices.	The number of client users associated with the Windows platform is significant. It provides wide browser variety, including Chrome, Edge, and Firefox browsers. Supporting the Windows client will include the need to support multiple browsers and screen resolutions. Development costs will be moderate for the Windows platform, as there is no need for the end user to have a platform-specific client application apart from the browser.	In the case of mobile platforms considerations will be made due to the size of the screen being different. The application will be responsive in nature, ensuring that usability is properly addressed in different screen sizes. This will add more effort to the development in terms of considering optimization for the application in iOS and Android devices individually. However, as the application will be browser-based, native applications will not be created, hence the cost will be minimized in the future.
--------------------	---	---	--	---

Development Tools	Various tools are supported for building different applications that are web-based, such as Java, JavaScript, and Integrated Development Environments such as Eclipse, IntelliJ IDEA, and Visual Studio Code. These tools will assist in simplifying the process of building server-based applications due to the Unix-based environment. In terms of licensing, costs are generally low. However, costs related to hardware can be high due to products from Apple Computer. A development group will assist in building as well as testing the application that supports clients running on macOS.	Linux fully supports development tools and programming languages that are used in web development, which are normally open source, such as Java, JavaScript, and even containerization tools such as Docker. IDEs such as Eclipse and VS Code are fully supported and free to use as well. The license cost for Linux is also low, making Linux a very cost-effective option for development as well as production. Linux development environments are also great for server-side development with CI/CD pipelines, for which fewer teams are required.	Windows supports a wide array of development tooling, including Java-based IDEs and web development frameworks. Integrations with enterprise tooling and services from Microsoft can be valuable in some cases. On the other hand, some of the development tooling and server configurations can include licensing costs. Additional configuration may be required by the development teams for parity with Linux-based production environments, which could increase the complexity in setting up the system.	The development tools involved in the development of the mobile device access may include front-end Web technology such as HTML, CSS, and JavaScript, and responsive design frameworks, etc. The only additional cost involved may be for the tools and expertise required in the testing phase, and no additional development tools would be required for the maintenance of the browser-based client, and hence no additional cost involved in the maintenance of the native client for iOS and Android devices.
--------------------------	---	--	---	---

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:**

The Gaming Room should consider implementing *Draw It or Lose It* on a system with Linux operating system. The Linux operating system is the most popular for web-based distributed and cloud-based applications. It offers high stability, strong load performance and support for open-source web servers, application servers, databases and containerization technologies. As most Linux distributions are free or very low cost, the licensing overhead is much reduced compared to Windows Server. Furthermore, the vast majority of cloud providers, AWS, Azure, Google Cloud and the like, support Linux virtual machines and containers as a first-class operation system choice. So, Linux is likely to be a more scalable and economic solution in the long run.

2. **Operating Systems Architectures:**

Linux has a modular monolithic kernel architecture. Though acting in similar roles, it is classified as a monolithic kernel, which can be independently loadable modules to allow easy swapping of multiple drivers. It integrates the operating system with process scheduling, memory management, file system, networking and device drivers.

Such an architecture is suitable for web-based game like *Draw It or Lose It* since it has high efficiency in system call handling, good isolation among processes and reliable networking stacks. Linux naturally supports multiuser and multitasking environments, which is necessary for being able host thousands of game sessions at once. Its process handling is fair and CPU time among threads and services isn't hijacked by multiplayer matches while games are responsive, it's able to render images on time even under load.

Additionally, Linux fits well with containerization tools like Docker and orchestration system such as Kubernetes. This means that applications can be spread out across multiple servers, so you can scale horizontally by running copies of app instances. This is a necessary ability in the distributed gaming world.

3. **Storage Management:**

For the maintainability of storage, it is preferable for the Linux setting to use a journaling file system like ext4 or XFS. For journaling file systems, metadata changes are journaled before committing them to avoid corruption in case of crashes and loss of power.

This can result in around 200 high-definition photographs, 1.6 GB estimated total, which are to be kept on redundant/persistent storage including but not limited to, RAIDX disk storage or cloud block storage. RAID technology enhances fault tolerance & read performance by writing data across multiple drives.

More importantly, the system must persist to disk in a relational database, like Postgres or MySQL, all of game data including users, sessions and scores. Static image files could be hosted on storage and delivered using a content delivery network (CDN) to help minimize server load and improve performance for global access.

4. **Memory Management:**

Linux has paging and demand loading with virtual memory management. In the clouds Virtual memory also comes into play allowing every process to have its own protected address space so that running game sessions don't tread on each other. Paging allows the OS to swap out inactive memory pages to disk as needed, keeping the system from crashing because of running out of memory.

Memory is a big concern for *Draw It or Lose It* since there will be multiple sessions running on the server at the same time that contain loaded images, ticking timers and player states. Linux's memory management system supports shared libraries, memory mapping, and caching mechanisms that improve performance. Frequent images, those that experience high accesses, are cached in memory to reduce the cost of disk access latency while unused resources are evicted by means of memory management policies.

As the game is time sensitive, it's important to cache disk access as much as possible when playing a round. Linux's buffer cache and page cache lower the number of disks reads, so your gaming images render more rapidly.

5. **Distributed Systems and Networks:**

The system needs to have client-server distributed model guidelines in order to serve conversation on desktop and mobile applications. Back-end Web application will run on a Linux server and perform RESTful API requests over HTTPS. Clients ,desktop browsers, Android, iOS, will connect to the server over the internet via standard protocols available in TCP/IP networking.

In distributed deployment, there could be multiple instances of the server running behind the load balancer. The server load balancer is responsible for routing the requests from the users to the servers and also ensures that no server is handling too many requests. If a server instance goes down, the requests can be routed to other instances.

However, in distributed systems, you have dependencies like network connectivity and the risk of outages. To counteract this vulnerability, the system must have:

- Health monitoring for server instances
- Automatic failover mechanisms
- Centralized database management
- Stateless application design where possible

Because you keep your application servers stateless and store session data in a central database or distributed cache, scale out is simple without any issues.

6. **Security:**

Security needs to safeguard user data at rest and in motion. Linux has a robust user and permission models to separate system services and block improper access. Key security measures should include:

- HTTPS connection with TLS to secure data in transit
- Storing sensitive user data in an encrypted database
- Permission Management where applicable (role based) for administrative tasks
- Fire wall configuration to minimize open port number
- Regular security patching and updates

There's also the fact that you shouldn't be storing passwords in plaintext and using a proper hashing algorithm like bcrypt or Argon2 for password storage instead. Server hardening practices, such as removing unnecessary services and employing intrusion detection systems, also lessen the exposure.

Authentication and session management are very important as the system is web based available on both mobile and desktop. Token based authentication, like JWT, can manage user sessions securely in distributed system.